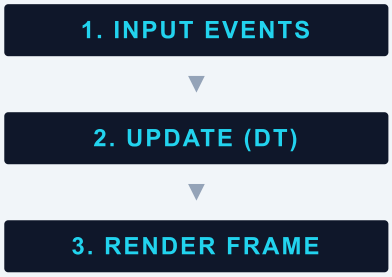
STAR DEFENDER ENGINE

ARCHITECTURE MATRICES & TECHNICAL GUIDE • EF101-P05

The Digital Flipbook Model: Star Defender operates with absolutely zero external multimedia assets (no images or audio files). The architecture relies entirely on mathematical procedural generation. The engine calculates vector geometries, synthesizes raw PCM audio frequencies in physical memory, and completely redraws the visual canvas 60 times every second to create the illusion of seamless interaction. This document breaks down the core structural pillars necessary for the system to compile and run.



MATRIX I: CORE ENGINE FUNDAMENTALS

SYSTEM COMPONENT	ARCHITECTURAL ROLE & FUNCTIONALITY	CODE LOGIC / SYNTAX

The Game Loop	The infinite `while` loop that acts as the heartbeat of the application. It forces the engine to listen for keystrokes, calculate physics, and render graphics in a rigid sequence.	`while running:`
		`  handle_events()`
		`  update()`
		`  draw()`

Delta Time (dt)	Hardware normalization protocol. Prevents the game from running faster on powerful CPUs by scaling all physical movement strictly to the fraction of a second passed.	`dt =`
		`clock.tick(60)/1000`
		`pos += speed * dt`

Engine Canvas	Maps logical entities to physical pixels. PyGame reverses standard math quadrants: the origin (0,0) is at the top-left, meaning the Y-axis grows positively downwards.	`Top-Left: (0,0)` `Move Down: y += n`

AABB Collisions	Axis-Aligned Bounding Box math. The engine checks if the rectangular boundaries of two independent objects overlap in 2D space to trigger damage or explosions.	`if`
		`A.rect.colliderect(B):`
		`  A.destroy()`

MATRIX II: OBJECT-ORIENTED BLUEPRINTS

CLASS ENTITY	ENCAPSULATED RESPONSIBILITY	PRIMARY DEPENDENCIES
Game	The Director. Manages the global state machine (Menu, Playing, Game Over), owns the display surface, and orchestrates the updates of all child entities.	`pygame.display` `pygame.event`
Player	Reads the keyboard input matrix every frame. Self-manages its own screen boundaries, firing cooldown timers, invulnerability frames, and live counts.	`pygame.key.get_pressed()`
Enemy	Instantiated procedurally. Tracks its own speed, health points, and movement AI (e.g., straight down, zigzag paths, or hovering gunners).	`random.uniform()` `math.sin()`
Projectile	Short-lived objects that travel along a single vector. Handles its own culling if it travels outside the engine canvas to prevent memory leaks.	`pygame.Rect` `self.y -= speed * dt`

MATRIX III: PROCEDURAL LOGIC & PHYSICS

MATHEMATICAL SYSTEM	IMPLEMENTATION & PURPOSE	CODE LOGIC / SYNTAX
Vector Graphics	Ships are drawn using `pygame.draw.polygon`. A list of coordinate tuples relative to a central `(x,y)` point is dynamically recalculated every frame to render the object.	`pygame.draw.polygon( surface, color, points)`
Audio Synthesis	Instead of `.mp3` files, raw byte arrays of sine/square waves are generated mathematically at runtime and loaded directly into memory for instant playback.	`buffer = numpy.sin(freq)` `sound = pygame.mixer.Sound()`
Sine Wave AI	Level 2 adversaries use a trigonometric sine wave to calculate their X-coordinate based on their current Y-coordinate, creating a smooth, oscillating flight path.	`x = start_x + sin(y) * amp`

MATHEMATICAL SYSTEM	IMPLEMENTATION & PURPOSE	CODE LOGIC / SYNTAX
Hitbox Reduction	To make the game feel "fair", physical collision boundaries are mathematically shrunk by 15% compared to the visual drawn shape, allowing for near-misses.	`rect.inflate(-4, -4)`

MATRIX IV: ADVANCED DATA INTEGRITY

INTEGRITY MECHANISM	PROBLEM SOLVED / CONCEPT	CODE LOGIC / SYNTAX
Atomic Saving	Prevents JSON corruption. If a PC crashes mid-save, the file is destroyed. Atomic saving writes to a temporary file first, only overwriting the master file if successful.	`write to temp.json` `os.replace(temp, master)`
List Culling	Prevents RAM bloat (Memory Leaks). If bullets fly off-screen forever, memory fills up. The engine scrubs arrays every frame to delete inactive objects safely.	`arr = [e for e in arr if e.active]`
Graceful Degradation	If a school lab computer has no audio device, initializing the mixer will crash the app. The system catches this exception and safely disables audio flags instead of crashing.	`try: mixer.init()` `except: audio_enabled = False`
Vector Normalization	Moving diagonally combines X and Y speeds, making the player faster. Normalization mathematically limits the diagonal vector length to equal cardinal speed.	`length = math.hypot(dx, dy)` `dx /= length`